Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Ders büyük olasılıkla 19 Nisan 2023 Çarşamba günü, Ramazan Bayramı'ndan önce online yapıldı. Vizeden bir hafta önceki uygulama dersiydi.
- Dersi ders asistanı Elçin Hanım anlattı. Prof. Dr. Oya Kalıpsız dersi açtı ve sözü asistana bıraktı.
- Örnek vaka: bir polikliniğin hasta ve çalışan yönetim yazılımı.
- Violet UML Editor ile üç çizim yapıldı:
  - Kullanım şeması (use case diagram): aktörler, etkileşimler, `include`, `extend`, aktör genelleştirmesi.
  - İki kullanım senaryosu: "Tedavi Bilgisi Girişi" ve "Ücret Ödeme".
  - Sınıf şeması (class diagram): kalıtım, birliktelik (association), aggregation, composition, çokluk (multiplicity).
- Vize sınavının biçimi duyuruldu.

Hocanın Özellikle Vurguladığı Kısımlar

- **Vize:** Gelecek hafta, ders gününde ve ders saatinde online yapılacak. Sorular test biçiminde olacak, ama şema çizdiren sorular da gelebilir.
- **Kullanım senaryosunda mutlaka olması gerekenler:** birincil aktör, ilgililer ve beklentileri, ön koşul, son koşul, ana akış ve alternatif akışlar.
- **extend, nesneye yönelik programlamadaki kalıtım değildir.** Yalnız bir işin bazı koşullarda genişlediğini, alternatif bir akışa girdiğini gösterir.
- **Alternatif akışta yazılan durumlar, kullanım şemasında `extend` olarak da gösterilmelidir.** Asistan, özel rapor gereken reçete durumunu örnek verdi.
- **Analist bakış açısıyla sınıf şemasında metotlar yazılmadı.** Yalnız sınıf adları ve nitelikler gösterildi.
- **İlişkiden anlaşılan bilgiyi nitelik olarak tekrar yazmaya gerek yok.** Asistan buna "gizli bilgi" (hidden information) dedi. Hasta ile Erişim arasında birebir ilişki çizildiyse, hasta sınıfına ayrıca "erisim" niteliği yazılmayabilir.
- **Aggregation ile composition farkı:** Parça, bütün silinince de var olmaya devam ediyorsa aggregation kullanılır. Bütün olmadan parça da olamıyorsa composition kullanılır.

Kısa Tekrar Notları

- **Aktör:** Sistemi kullanan kişi ya da dış sistem. Çöp adamla çizilir.
- **Etkileşim (interaction):** aktör ile kullanım senaryosu arasındaki düz çizgi.
- **include:** Bir senaryonun gerçekleşmesi için başka bir senaryo da çalışır.
- **extend:** Bir senaryonun bazı koşullarda genişleyen, alternatif hâli.
- **Aktör genelleştirmesi (generalization):** Özel aktör, genel aktörün yaptığı her şeyi yapar ve kendine özgü işleri de vardır. Örnek: Başhekim bir Hekim'dir.
- **Sınıf kutusunun üç bölümü:** ad, nitelikler (attributes), metotlar.
- **Görünürlük:** - private, + public.
- **Kalıtım (inherits):** Ortak nitelikler üst sınıfta bir kez tanımlanır.
- **Çokluk:** 1 tam olarak bir, * sıfır ya da daha çok, `1..*` bir ya da daha çok.
- **Aggregation:** bütün tarafında içi boş elmas. **Composition:** bütün tarafında içi dolu elmas.

Detaylı Açıklamalar

Vaka metni: poliklinik yönetim yazılımı

- **Çalışan bilgileri:** ad, soyad, kimlik numarası, işe başlama tarihi, çalıştığı birim, ayrıldıysa ayrılma tarihi.
- **İletişim bilgileri:** ikamet adresi, iş ve cep telefonu.
- **Çalışan türleri:** doktor, hemşire, hizmetli ve güvenlik görevlisi. Doktorlar için ayrıca diploma numarası tutulur.
- **Doktorlar:** uygun günlerini, randevularını ve hastalarının tedavisini yönetir; reçete yazar, tedavi bilgisi girer.
- **Başhekim:** Bir doktordur. Hastalarla ilgilenmenin yanında hastane yönetiminden de sorumludur.
- **Hasta bilgileri:** ad, soyad, kimlik numarası, adres, telefonlar.
- **Hastalar:** sisteme girip randevu alır. Randevu günü bankoda ücreti öder. Hastanın çalıştığı kurumun poliklinikle anlaşması varsa ücrette indirim yapılır.
- **Kayıt görevlisi:** Ödemeden sonra hastanın kaydını açar. Hasta muayene sırasına girer.
- **Hasta kaydında tutulanlar:** başvurulan birim, tedavinin başlangıcı, doktor, tanı, ilaçlar.

1. Kullanım şeması

Asistanın çizdiği şemanın metin hâli:

```
Hasta --------- (Randevu Alma)
Hasta --------- (Tedavi Bilgileri Görüntüleme)
Hasta --------- (Ücret Ödeme)
Kayıt Görevlisi --- (Hasta Kaydı)
(Hasta Kaydı) ----<<include>>----> (Ücret Ödeme)
(Anlaşmalı Kurum İndirimi) ----<<extend>>----> (Ücret Ödeme)

Hekim --------- (Hasta Bilgisi Görüntüleme)
Hekim --------- (Tedavi Bilgisi Girişi)
Hekim --------- (Randevu Görüntüleme)
(Tedavi Bilgileri Görüntüleme) ----<<include>>----> (Tedavi Bilgisi Girişi)
(Yatarak Tedavi) ----<<extend>>----> (Tedavi Bilgisi Girişi)

Başhekim ----|> Hekim                       (aktör genelleştirmesi)
Başhekim --------- (Planlama Yapma)
Başhekim --------- (Çalışan Bilgisi Görüntüleme)

Hekim --------- (Beyaz Kod Verme) --------- Güvenlik Görevlisi
```

Gerekçeler:

- **Hasta Kaydı → Ücret Ödeme (include):** Asistana göre kayıt açılabilmesi için ödeme önce yapılmış olmalı. (Bu bir ön koşuldur; `include` ile değil, senaryonun ön koşul alanıyla gösterilmeli, bkz. ders sonrası notlar.)
- **Anlaşmalı Kurum İndirimi → Ücret Ödeme (extend):** Ödeme normalde sabit ücretle yapılır. Kurum anlaşması varsa indirimli ödeme alternatif bir akıştır.
- **Yatarak Tedavi → Tedavi Bilgisi Girişi (extend):** Doktor hastanın durumunu ağır bulursa yatarak tedaviye karar verebilir.
- **Başhekim → Hekim (genelleştirme):** Başhekim, hekimin yaptığı her şeyi yapar. Ek olarak yönetim işleri vardır.
- **Beyaz kod:** Hasta ya da hasta yakını güvenliği tehdit ederse hekim beyaz kod verir. Güvenlik görevlisi kodu alır ve müdahale eder. (Şemada tek bir senaryonun iki aktöre bağlandığı altyazıdan kesin anlaşılmıyor.)

2. Kullanım senaryosu: Tedavi Bilgisi Girişi

- **Birincil aktör:** Hekim.
- **İlgililer ve beklentileri:**

  - Hekim: hasta bilgilerine ulaşmak ve kararlaştırdığı tedavinin bilgilerini sisteme girmek ister.
  - Hasta: bilgilerini görmek ve reçetesine ulaşmak ister.
- **Ön koşullar:**
  - Hekim kendi kullanıcı adı ve şifresiyle sisteme giriş yapmıştır.
  - Hekim hastanın kimlik bilgisini girerek hasta ekranına ulaşmıştır.
- **Son koşullar:** Reçete oluşturulmuştur. Girilen tedavi bilgileri kaydedilmiştir.
- **Ana akış:**
  - 1. Hekim hasta için "tedavi bilgisi girişi" işlemini seçer.
  - 2. Hekim teşhis edilen hastalığı girer.
  - 3. Hekim reçeteyi hazırlar: ilaçları ve dozlarını girer.
  - 4. Sistem reçeteyi onaylar ve reçete hastaya verilir.
- **Alternatif akışlar:**
  - 2a. Hastanın durumu ağırdır: Hastanın yatarak tedavi alması kararlaştırılır.
  - 4a. Reçetedeki ilaçlardan biri ya da birkaçı özel rapor gerektiriyordur: (1) Hekim raporu hazırlar. (2) Hekim raporu sisteme kaydeder.

Onay adımının amacı, ilaçlar için rapor gerekip gerekmediğini sistemin bildirmesidir. Bir adım için birden çok farklı durum varsa 2a, 2b, 2c diye numaralanır. Her durumun altına yapılacak işler sırayla yazılır. Tek adımda bitiyorsa alt madde açmaya gerek yoktur.

3. Kullanım senaryosu: Ücret Ödeme

- **Birincil aktör:** Hasta.
- **İlgililer ve beklentileri:**
  - Hasta: ödemesini yapıp muayene olmak ister.
  - Kayıt görevlisi: ödemeyi alıp sisteme işlemek ister.
- **Ön koşullar:** Hasta randevu almıştır. Hasta randevu günü polikliniğe gelmiştir.
- **Son koşullar:** Ödeme başarıyla alınmış ve fatura hazırlanmıştır. Hasta muayene sırasına alınmıştır.
- **Ana akış:**
  - 1. Sistem, hastanın aldığı randevuya göre ödenecek tutarı gösterir. Tutar, doktorun unvanına göre değişebilir.
  - 2. Hasta ödemeyi yapar.
  - 3. Sistem faturayı oluşturur.
- **Alternatif akışlar:**
  - 2a. Hasta poliklinikle anlaşmalı bir kurumda çalışıyordur: Belirlenen indirim oranı uygulanarak ücret alınır.
  - 2b. Kartla ödemede sorun yaşanır: Ödemenin nakit yapılması istenir.

4. Sınıf şeması

Asistanın çizdiği şemanın metin hâli:

```
Erisim:  -ad: String, -soyad: String, -kimlikNo, -telefonlar: String[], -adres: String

Calisan: -gorevBaslamaTarihi, -gorevdenAyrilmaTarihi, -erisim: Erisim
   Calisan 1 ---- 1 Erisim
   Hekim, Hemsire, GuvenlikGorevlisi, Hizmetli  --|>  Calisan   (kalıtım)
   Hekim: -diplomaNo: int
   Bashekim --|> Hekim

Hasta 1 ---- 1 Erisim
Hasta 1 ---- * Randevu

Recete: -no, -duzenleyen: Hekim, -tarih, -hasta: Hasta, -ilaclar: Ilac[]
   Hasta 1 ---- * Recete
   Hekim 1 ---- * Recete
   Recete <>---- Ilac          (aggregation, 1'e çok)
Ilac: -ad: String
```

```
Hastane: -ad, -kurulusTarihi
   Hastane <#>---- Bolum       (composition, 1'e çok)
Bolum: -bolumAdi: String, -calisanlar: Calisan[]
   Bolum <>---- Calisan        (aggregation)
```

Tasarım kararları:

- **Erişim bilgisi ayrı sınıfta:** Ad, soyad ve iletişim bilgileri hem çalışanda hem hastada tutulur. Bu bilgiler tekrar yazılmasın diye ayrı bir Erişim sınıfına konur. Çalışan ve Hasta bu sınıfla birebir ilişki kurar. Telefonlar birden çok olabildiği için dizi olarak tutulur.
- **Kalıtım:** Dört çalışan türü ortak nitelikleri Çalışan sınıfından alır. Yalnız Hekim'in ek bir diploma numarası vardır. Başhekim'in ayrı işlemleri olduğu için ayrı bir sınıf olarak Hekim'den türetilir.
- **Reçete–İlaç (aggregation):** İlaçlar bir reçetede bir araya getirilir. Reçete silinse de ilaç tanımları var olmaya devam eder.
- **Hastane–Bölüm (composition):** Hastane olmadan bölüm olamaz. Hastane silinirse bölümleri de silinir.
- **Bölüm–Çalışan (aggregation):** Bölüm kapanırsa çalışan başka bölüme aktarılabilir. Çalışan tek başına var olabilir.
- **Gizli bilgi:** Hastane'nin bölümler dizisi şemada nitelik olarak yazılmadı. Composition ilişkisi bunu zaten gösteriyor.

Ders sonrası notlar

- **`include` ön koşul anlamına gelmez.** Asistan "kayıt açılması için önce ödeme yapılmış olmalı" ve "tedavi bilgisi görüntülemek için önce giriş yapılmış olmalı" gerekçesiyle `include` kullandı. UML'de `include`, temel senaryo her çalıştığında içerilen senaryonun da o anda çalışması demektir. Ortak bir adımı ayırmak için kullanılır. Zaman sırası ya da ön koşul `include` ile gösterilmez; senaryonun ön koşul alanına yazılır.
  - Tedavi bilgilerini görüntülemek, tedavi bilgisi girmeyi çalıştırmaz. Bu ikisi arasında `include` olmamalıdır. "Tedavi bilgisi girilmiş olmalı" görüntüleme senaryosunun ön koşuludur.
  - Hasta Kaydı ile Ücret Ödeme ayrı aktörlerin ayrı işleridir. Ödemenin önce yapılmış olması, Hasta Kaydı senaryosunun ön koşulu olarak yazılmalıdır.
- **Okların yönü:** `include` oku temel senaryodan içerilen senaryoya gider. `extend` oku genişleten senaryodan temel senaryoya gider. Derste çizilen oklar bu yöndedir.
- **`*` çokluğu.** Asistan * için "bir veya birden çok" dedi. UML'de *, `0..*` demektir: sıfır ya da daha çok. "En az bir" için `1..*` yazılır.
- **İlaç dozu nerede tutulur?** Doz bir ilacın değil, o reçetedeki kalemin özelliğidir. Aynı ilaç farklı reçetelerde farklı dozla yazılır. Daha doğru bir model, Reçete ile İlaç arasına doz ve kullanım bilgisini tutan bir ReçeteKalemi sınıfı koymaktır. Bu, Ders 6'daki SatışKalemi–Ürün ilişkisine benzer.
- **Diploma numarası tipi.** Diploma ve kimlik numaraları hesaplamada kullanılmaz ve başında 0 ya da harf olabilir. Bu yüzden `int` yerine `String` tutulması daha güvenlidir.
- **"Gizli bilgi" terimi.** Asistanın "hidden information" dediği şey yerleşik bir UML terimi değildir. UML'de bir ilişki, uçtaki rol adıyla zaten bir niteliği temsil eder. Aynı bilgiyi hem ilişki hem nitelik olarak yazmak gereksiz tekrardır. Asistanın önerdiği gibi tek biçimde göstermek yeterlidir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.